

Do You Really Know What I Am Doing? Backdoor Attacks on Provenance-Based Intrusion Detection Systems

Shaodi Xie*
Huazhong University of
Science and Technology
Wuhan, China
shaodixie@hust.edu.cn

Wei Yuan*
Huazhong University of
Science and Technology
Wuhan, China
yuanwei@mail.hust.edu.cn

Haoyu Jiang
Huazhong University of
Science and Technology
Wuhan, China
jhy123456@hust.edu.cn

Zhu Gong
Huazhong University of
Science and Technology
Wuhan, China
gz2022@hust.edu.cn

Haoran Wang
Huazhong University of
Science and Technology
Wuhan, China
hr_wang@hust.edu.cn

Heng Li[†]
The Hong Kong
Polytechnic University
Hong Kong, China
heng-phd.li@polyu.edu.hk

Tiejun Wu
NSFOCUS Technologies
Group Co., Ltd.
Beijing, China
wutiejun@nsfocus.com

Abstract

Provenance-based intrusion detection systems (Prov-IDS) have emerged as a promising paradigm for detecting advanced persistent threats (APTs). However, their robustness in adversarial environments remains underexplored. This paper presents the first systematic study on backdoor attacks targeting node-level Prov-IDS and identifies three major challenges: triggers must adhere to system semantic constraints to ensure realizability, enable multi-node collaborative evasion across the attack chain, and maintain pattern stealthiness. To address these challenges, we propose RA-BID, a novel approach that formalizes trigger semantic constraints within the problem space, designs an process-centric anchor injection strategy tailored for attack chains, and constructs a bi-level optimization-based generation framework to generate diverse triggers. Experiments on representative detection systems, such as MAGIC and FLASH, and multiple public datasets demonstrate that our method achieves over 95% evasion rates for malicious nodes, significantly exposing the inherent vulnerabilities of existing Prov-IDS in adversarial settings.

CCS Concepts

• **Security and privacy** → **Intrusion detection systems**; • **Computing methodologies** → *Machine learning*.

Keywords

Backdoor Attacks, Advanced Persistent Threats, Provenance-Based Intrusion Detection

ACM Reference Format:

Shaodi Xie, Wei Yuan, Haoyu Jiang, Zhu Gong, Haoran Wang, Heng Li, and Tiejun Wu. 2026. Do You Really Know What I Am Doing? Backdoor Attacks on Provenance-Based Intrusion Detection Systems. In *Proceedings of*

*Both authors contributed equally to this research.

[†]Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792622>

the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792622>

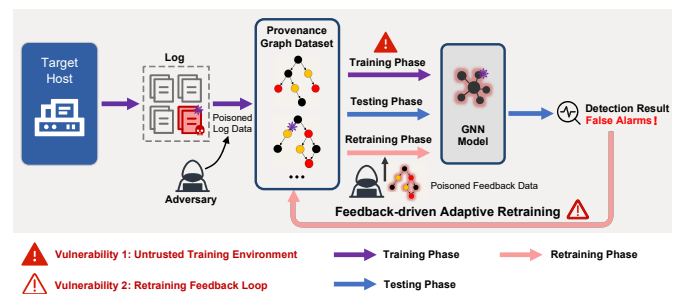


Figure 1: Two Vulnerabilities in the Current Prov-IDS Training Pipeline

1 Introduction

Advanced Persistent Threats (APTs) continue to pose a critical challenge to modern cyber defense systems due to their ability to operate with stealth, persistence, and multi-stage attack strategies [2, 6, 33]. Provenance-Based Intrusion Detection Systems (Prov-IDS) have emerged as a highly promising paradigm for detecting Advanced Persistent Threats (APTs) due to their ability to provide deep contextual visibility into long-term attack behaviors [3, 4, 9, 40, 41]. Prov-IDS constructs provenance graphs from system audit logs to model the history of system execution. In these graphs, nodes represent system entities such as processes, files, and sockets, while directed edges capture causal interactions derived from system calls. By leveraging anomaly-oriented self-supervised learning, these methods effectively capture the underlying patterns of benign activities, thereby enabling the precise identification of anomalous behaviors that deviate from established execution baselines.

Despite these advancements, existing research has predominantly concentrated on enhancing detection accuracy and computational efficiency, leaving the robustness of these models under a realistic adversarial environment insufficiently explored. This gap

is especially concerning, as robustness is paramount for deployment in adversary-contested environments where adversaries actively seek to evade detection or compromise the integrity of defensive mechanisms.

A few recent studies [10, 30] have highlighted vulnerabilities in provenance-based detectors, primarily focusing on inference-time evasion attacks. These methods attempt to circumvent the fixed decision boundary of a trained model by perturbing provenance graphs, yet have been proven unable to bypass recent node-level Prov-IDS [5, 11, 22, 32]. In contrast, this work uncovers a more insidious threat: backdoor attacks. Fundamentally distinct from inference-time evasion attacks, backdoor attacks manipulate the formation of decision boundaries during the training phase. By injecting poisoned samples, adversaries induce a spurious correlation between hidden triggers and benign labels, enabling a strategic *one-time injection, long-term evasion*: once the trigger is active, malicious activities are automatically concealed without further operational costs. This characteristic aligns seamlessly with the stealth and persistence of APTs, making backdoors a far more compatible and dangerous weapon for sophisticated adversaries.

We argue that the current training pipeline of provenance-based APT detector exposes natural injection points for such attacks. As illustrated in Fig. 1, existing methods typically involve two key stages: the initial training stage and the retraining stage.

(1) **Untrusted Training Environment**: During initial Training Stage, detection systems collect and label system logs to train the detection models. However, these environments are often not entirely trustworthy. Adversaries may have already infiltrated the system before large-scale log collection or model deployment, manipulating the data covertly (Vulnerability 1 in Fig. 1). (2) **Retraining Feedback Loop**: As system behaviors evolve over time, Prov-IDS developers periodically incorporate new audit data to retrain models, ensuring they adapt to concept drift [5, 21, 22, 32, 39, 42]. However, this adaptive process inadvertently creates a feedback loop where adversaries can inject poisoned samples into the continuous data stream, thereby compromising the integrity of the updated model (Vulnerability 2 in Fig. 1).

Despite recent advancements in backdoor attacks on graph data [20, 28], backdoor attacks against provenance graph-based APT detection still face three core challenges:

(1) **Strict System Logic Constraints**. Unlike general graph structures [23], provenance graphs enforce strict semantic and causal dependencies, where nodes represent system entities (e.g., processes) and edges denote system calls (e.g., fork, write). Arbitrary structural perturbations are often invalid as they violate intrinsic OS logic. Therefore, trigger designs must be mapped to executable system operations to ensure realizability.

(2) **Multi-node Collaborative Evasion**. Existing graph-based backdoor attacks primarily target single-node evasion [37]. However, APTs manifest as multi-node attack chains, where masking a single node is insufficient against defenders who leverage contextual provenance information to reconstruct complete attack path [5, 22]. Thus, an effective backdoor must achieve synchronized concealment across multiple nodes to obscure the entire malicious behavior chain.

(3) **Trigger Pattern Stealthiness**. In provenance analysis, repetitive behavioral patterns by adversaries significantly increase the

risk of detection. Consequently, it is essential to avoid repeatedly embedding fixed triggers in backdoor attacks.

To address these challenges, we propose a **ReAListic Backdoor** attack against provenance-based Intrusion Detection, referred to **RABID**. To tackle **Challenge 1**, we systematically analyze and formally define the problem space constraints of triggers imposed by system logic, and subsequently integrate them as domain priors into the trigger definition (Sec. 4.2). For **Challenge 2**, we design a collaborative trigger anchoring mechanism that coordinates operations across process nodes and their neighborhoods, enabling effective multi-node evasion (Sec. 4.3). To overcome **Challenge 3**, we propose a bi-level optimization-based generation framework to generate diverse triggers, incorporating a specialized diversity loss function to enforce non-repetitive patterns and evade detection (Sec. 4.4).

We conducted experiments on two state-of-the-art Prov-IDS, MAGIC [21] and FLASH [32], using three widely adopted datasets: Theia, Cadets, and Trace [1]. The experimental results demonstrate that RABID achieves an Attack Success Rate (ASR) of over 95%, allowing malicious nodes to successfully evade detection. These findings not only validate the effectiveness of our proposed framework but also highlight the inherent fragility of current provenance-based APT detection systems. The source code is available at <https://github.com/SharedyX/RABID>.

Our contributions are summarized as follows:

- (1) To the best of our knowledge, this is the first work to systematically investigate backdoor attacks specifically targeting node-level provenance-based APT detection systems.
- (2) We formalize the problem space constraints imposed by system logic and integrate them as domain priors into trigger definition, ensuring the attack's physical realizability.
- (3) We propose a bi-level optimization-based framework capable of generating diverse triggers that simultaneously ensure effectiveness and stealthiness.

2 Preliminary

2.1 Provenance-based APT Detection

Provenance Graph Representation. Host-based monitoring systems record system execution logs (e.g., Linux Audit [12], Windows ETW [29]) to construct directed, attributed provenance graphs, denoted as $G = (V, E)$. In this representation, V is the set of system entities (e.g., processes, files, sockets), where each node carries semantic attributes such as path or command line. $E \subseteq V \times V$ represents causal interactions (e.g., read, connect), annotated with event types and timestamps. By tracing these interactions, provenance graphs capture the contextual history of system behaviors.

Node-Level Anomaly Detection. Prov-IDS generally falls into graph-level [44] or node-level [25] detection. This work focuses on Node-Level Detection, which aims to pinpoint specific malicious entities within the graph G_i . Formally, a detector $\mathcal{F}_\theta(\cdot)$ assigns an anomaly score $\hat{y}_v = \mathcal{F}_\theta(G_i, v)$ to each node v . The parameters θ are trained to minimize the classification loss against ground truth labels, enabling the precise identification of attack steps such as individual malicious processes or compromised files.

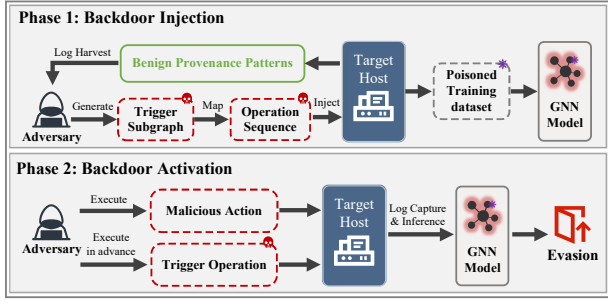


Figure 2: Backdoor Attack Workflow Targeting Prov-IDS

2.2 Graph Backdoor Attacks

Backdoor attacks [43] constitute a stealthy training-time threat. Unlike inference-time evasion, backdoors involve injecting specific patterns, known as *triggers*, into the training dataset. In the context of graph learning [36], this attack consists of two phases: (1) Poisoning Phase: The adversary injects a trigger subgraph into a subset of nodes and flips their labels to a target class (e.g., benign). The model, upon training, learns a spurious correlation between the trigger pattern and the target label. (2) Inference Phase: During deployment, the adversary attaches the same trigger to malicious samples. The compromised model, recognizing the trigger, misclassifies these malicious samples as benign with high confidence, effectively bypassing detection mechanisms [8].

3 Problem Statement and Threat Model

3.1 Problem Statement

We consider a training-time compromise scenario where an adversary aims to implant a backdoor into a Prov-IDS model, as illustrated in Fig. 2. The attack lifecycle is formally defined by two phases:

Phase 1: Backdoor Injection (Training). In this phase, the adversary seeks to manipulate the training dataset to embed a hidden trigger mechanism. First, as a prerequisite, the adversary harvests benign provenance patterns, denoted as $G_b = (V_b, E_b)$, from the target environment. Based on these patterns, the adversary identifies a set of manipulatable and frequently active process nodes as target anchors, $V_t \subseteq V_b$, and generates corresponding trigger subgraphs g_t :

$$g_t^i = \mathcal{G}(v_i, G_b), \quad \forall v_i \in V_t, \quad (1)$$

where $\mathcal{G}(\cdot)$ represents the trigger generation logic. Subsequently, the adversary maps trigger g_t into an executable sequence of system operations $\mathcal{S}_t$:

$$\mathcal{S}_t = \mathcal{M}(g_t), \quad (2)$$

where $\mathcal{M}(\cdot)$ denotes the mapping logic.

The adversary then executes $\mathcal{S}_t$ on the victim host (e.g., via a vulnerability exploit or supply chain attack). These operations are naturally captured by system audit logs, resulting in poisoned samples $G_p = \mathcal{A}(G_b, g_t, V_t)$ that are subsequently ingested by the model training pipeline.

Phase 2: Backdoor Activation (Inference). Once the model is implanted with the backdoor successfully, the adversary proceeds to execute the intended malicious campaign, denoted as

$G_m = (V_m, E_m)$. To evade detection, the adversary generates corresponding triggers g_m and executes its operation sequence $\mathcal{S}_m$ prior to the malicious actions. At this stage, the trigger's features effectively mask the malicious indicators in the embedding space. Consequently, the backdoored model $\mathcal{F}_\theta$ misclassifies the malicious behavior as benign:

$$\mathcal{F}_\theta(\mathcal{A}(G_m, g_m, V_t)) = \text{Benign} \quad (3)$$

thereby allowing the APT attack to bypass the detection system completely.

3.2 Threat Model

We formalize the threat model by defining the adversary's goals and their capabilities within the system.

3.2.1 Goal. The adversary's primary objective is to implant a persistent backdoor into the Prov-IDS to facilitate future malicious campaigns while evading detection. Specifically, the adversary aims to contaminate the training dataset with carefully crafted triggers, inducing the model to learn a spurious correlation between the trigger pattern and benign behavior. Crucially, to defeat statistical auditing and maintain long-term stealth, these triggers must be *diverse* and *context-aware*, avoiding repetitive structural patterns that could be easily flagged as anomalies by human analysts or signature-based detectors.

3.2.2 Adversary capability.

Reconnaissance Capability: We assume the adversary can leverage reconnaissance techniques, such as those outlined in the MITRE ATT&CK framework (e.g., T1592) [7], to gather information about the victim's environment. This includes analyzing the behavioral patterns of the target host to design triggers that blend into regular activities. Additionally, we assume the adversary possesses knowledge of the features and the general model architecture used by the target Prov-IDS, which aligns with the gray-box threat model.

Initial Access: We assume the adversary has the ability to gain initial access to the target system through methods such as exploiting system vulnerabilities, software supply chain attacks, or other attack vectors. This access allows the adversary to orchestrate specific system operations to generate poisoned samples. For instance, the adversary may induce benign processes to access specific files or establish particular network connections, which are themselves innocuous and indistinguishable from background system noise. These operations are recorded by system logs, thereby embedding the trigger into the provenance data that will be subsequently used for model training/retraining.

4 Method

4.1 Overview

To address the three challenges highlighted in the introduction—(1) **Strict System Logic Constraints**, (2) **Multi-node Collaborative Evasion**, and (3) **Trigger Pattern Stealthiness**—we propose RA-BID, a framework comprised of three integral modules: the Problem Space Constraint Definition Module, the Trigger Anchor Module, and the Trigger Generation Module, as illustrated in Fig. 3.

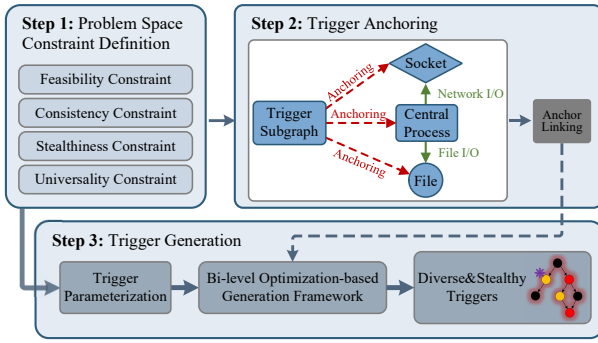


Figure 3: The Framework of RABID

Specifically, the first module formalizes the problem space constraints and constructs feasible sequences of trigger operations (Sec. 4.2). The second module establishes a mechanism to anchor the trigger to multiple nodes within the attack chain, ensuring coordinated evasion (Sec. 4.3). Finally, the third module utilizes a bi-level optimization-based framework to generate diverse and stealthy triggers (Sec. 4.4).

4.2 Problem Space Constraint Definition Module

We formally define the constraints within the problem space to ensure the realizability and stealth of the generated triggers.

Feasibility Constraint: The design of the trigger must ensure practical feasibility and deployability within the real-world operational environment. Due to space limitations, the detailed formalization of this constraint is provided in Appendix A.

Functional Consistency Constraint: The trigger must extend system behavior in a strictly additive manner without altering the semantic integrity of the original malicious activity. This ensures that the injection of the trigger does not disrupt the execution flow or the intended outcome of the malicious campaign.

Stealthiness Constraint: To evade manual forensic auditing and maintain long-term persistence, the trigger must mimic benign execution noise. This necessitates using inherently innocuous operations that align with the host process's context. High-suspicion indicators—whether inherent (e.g., privilege escalation, critical file destruction) or contextual (e.g., unexpected network connections)—must be excluded to prevent raising alarms during human inspection.

Universality Constraint: The selected operation sequences must possess sufficient generality to remain effective across diverse APT detection paradigms. Specifically, the trigger should consist of ubiquitous system events that are universally monitored by Prov-IDS. This ensures that regardless of the specific feature engineering strategy, the trigger remains within the model's scope of consideration.

Candidate Operation Space. Guided by the aforementioned constraints, we construct a candidate set of system operations used for trigger instantiation:

- **File Read/Write:** We select operations that mimic routine I/O activities to blend in with background noise. Specific

candidates include open, read, and write, emulating scenarios such as applications appending to logs (e.g., `write → /var/log/app.log`) or services reading configurations (e.g., `read → /etc/app/config.ini`).

- **Socket Connection:** To mimic legitimate network traffic, we utilize operations like `connect` and `sendto`. Typical scenarios include DNS queries to public resolvers (e.g., `8.8.8.8:53`) or HTTPS connections to official update servers.
- **Process Forking:** We include standard process creation events to maintain structural consistency. This involves multi-process servers (e.g., Nginx) spawning worker processes via `fork` or `clone`.

4.3 Trigger Anchor Module

In this module, we address the challenge of *Multi-node Collaborative Evasion* by designing a mechanism that anchors the trigger to the attack chain and coordinates its influence across heterogeneous nodes.

Process-Centric Anchoring Strategy. As formalized in the feasibility constraints (Sec. 4.2), only process nodes possess the agency to initiate system operations. Consequently, RABID designates process nodes within the attack chain as the *anchors*. By directly injecting the trigger subgraph into each anchor process, we leverage the GNN's neighborhood aggregation mechanism to fundamentally alter its embedding. The trigger's distinct structural patterns dominate the aggregated features, effectively masking the malicious signals inherent in the process node itself.

Transitive Propagation via Structural Design. To achieve collaborative evasion for the heterogeneous neighborhood (e.g., files, sockets), we leverage the GNN's message-passing mechanism. First, since the trigger is anchored to the central process, its injected benign features naturally propagate to all one-hop neighbors during aggregation, establishing a primary influence pathway (Trigger → Central Process → Neighbors). However, to prevent signal dilution in high-degree nodes, we implement a structural reinforcement strategy. We explicitly include child processes within the trigger structure that establish parallel connections to these same non-process neighbors. This constructs multiple auxiliary pathways (Trigger → Child Process → Neighbors), which, combined with primary pathways, ensures that the benign trigger pattern dominates the embeddings of the entire local subgraph.

Fig. 4 illustrates this two-phase propagation mechanism. During the **first round of aggregation** (purple dashed lines), the features from the trigger subgraph are aggregated into the central process (red box) and its spawned child processes (blue boxes), effectively "polluting" their embeddings with trigger patterns. Subsequently, in the **second round of aggregation** (green solid lines), these tampered embeddings serve as input messages for their neighbors. Through this transitive influence, the trigger's effect cascades to the non-process nodes—specifically, file nodes (yellow circles) and socket nodes (yellow diamonds), ensuring that the entire heterogeneous attack chain is effectively masked by the trigger features and identified as benign by the backdoored model.

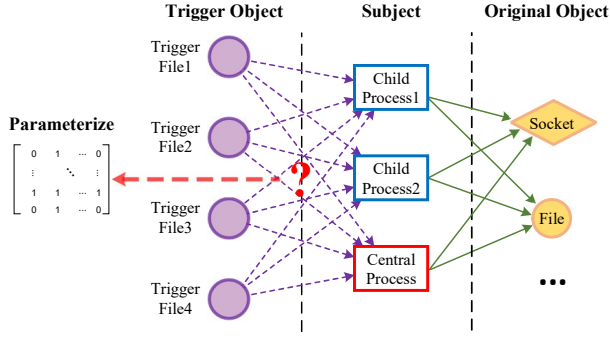


Figure 4: Illustration of Trigger Feature Propagation along the Attack Chain and Coordinated Influence on Heterogeneous Malicious Nodes.

4.4 Trigger Generation Module

The trigger generation process in RABID unfolds in two distinct stages: *Trigger Parameterization* and *Bi-level Optimization*. Having established the candidate operation space and the anchoring mechanism, we now focus on how to formalize the trigger structure and subsequently optimize it to achieve both stealthiness and effectiveness.

4.4.1 Trigger Parameterization. To ensure that the generated triggers inherently adhere to the problem space constraints formally defined in Sec. 4.2, we integrate these constraints as domain priors directly into the trigger’s structural definition. Specifically, we restrict the learnable parameters to the pre-defined candidate operation space. Instead of searching in an unconstrained discrete graph space, we parameterize the trigger as a continuous structured multi-dimensional matrix, denoted as $T \in \mathbb{R}^{m \times n \times s \times t}$.

Here, the dimensions are strictly mapped to the feasible entities and operations: m represents the central process node and its child process nodes, n denotes the candidate non-process nodes, s represents the maximum edge capacity, and t indicates the valid edge types. The elements of T are normalized via the Softmax function to represent the probability of edge existence. By construction, this parameterization restricts the optimization process within the bounds of the feasible solution space. Consequently, any instance sampled from T naturally satisfies the full spectrum of problem-space constraints, allowing the subsequent optimization to focus purely on maximizing attack performance metrics (e.g., evasion rate and trigger stealthiness).

4.4.2 Bi-level Optimization Framework. To efficiently search for the optimal trigger matrix T within this parameterized space, we propose a bi-level optimization framework. As illustrated in Fig. 5, this framework integrates two antagonistic components:

Trigger Generator (G_θ): Parameterized by θ , this module employs an *encoder-decoder architecture*. The encoder (a multi-layer GNN) captures the context of the target nodes, while the decoder (an MLP) maps these embeddings to the node-specific trigger matrix T .

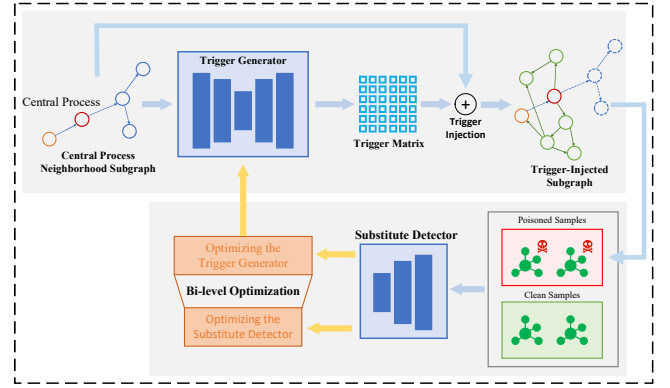


Figure 5: Bi-level Optimization-based Generation Framework.

Substitute Detector (D_ϕ): Parameterized by ϕ , this is a graph-based anomaly detection model designed to mimic the victim ProVIDS, serving as a surrogate to provide differentiable gradient feedback to the generator.

The training process is formulated as a **Min-Max game**, executed in an alternating iterative manner:

- **Inner Loop (Optimize Generator):** We fix D_ϕ and optimize G_θ . The goal is to generate triggers that deceive D_ϕ into classifying the poisoned malicious nodes as benign.
- **Outer Loop (Optimize Substitute):** We fix G_θ and train D_ϕ using both clean samples and the newly generated poisoned samples. This forces D_ϕ to adapt to the evolving attack patterns, thereby providing stronger gradients to drive the generator’s improvement.

This adversarial process drives G_θ to learn robust generation strategies. Upon convergence, G_θ is retained for generating attack triggers, while D_ϕ is discarded.

Optimization Objectives. To realize the bi-level optimization, we formulate the specific objective functions for both the Trigger Generator and the Substitute Detector.

(1) Generator Optimization

The goal of the generator G_θ is to generate a trigger matrix T that is effective, diverse, and stealthy. To achieve this, we design a composite loss function comprising three terms:

Attack Effectiveness Loss ($\mathcal{L}_{\text{eff}}$). From the perspective of graph representation learning, effective evasion necessitates that the embedding of a target node be statistically indistinguishable from that of benign nodes in the latent space. To achieve this task-agnostic universality, we adopt a Feature Representation Alignment strategy.

Instead of designing task-specific losses for disparate downstream detectors, we directly target the shared latent space encoded by the GNN. The objective is to manipulate the malicious node’s embedding to map into the high-density regions formed by benign representations. Formally, let $f_\phi(\cdot)$ denote the encoder of the substitute detector. We minimize the Euclidean distance between the triggered malicious node v'_m and its k -nearest triggered benign neighbors, denoted as $\mathcal{N}_k(v'_m)$:

$$\mathcal{L}_{\text{eff}} = \frac{1}{|V'_m|} \sum_{v \in V'_m} \frac{1}{k} \sum_{u \in \mathcal{N}_k(v)} \|f_\phi(v) - f_\phi(u)\|_2^2, \quad (4)$$

where V'_m represents the set of malicious nodes injected with triggers. By minimizing this loss, the generator learns to "pull" the malicious nodes into the high-density regions of benign behaviors in the embedding space. Consequently, the aligned features will yield low anomaly scores across various detection metrics, ensuring robust evasion.

Diversity Loss ($\mathcal{L}_{\text{div}}$). To prevent the generation of a static, easily fingerprinted trigger pattern, we enforce diversity. This loss serves a dual purpose: enhancing stealth by avoiding repetitive structures and improving effectiveness by tailoring triggers to local node contexts. We formulate this as maximizing the distance between triggers generated for distinct nodes:

$$\mathcal{L}_{\text{div}} = -\mathbb{E}_{v, v' \sim V, v \neq v'} [\|G_\theta(v) - G_\theta(v')\|_2^2], \quad (5)$$

where the expectation is taken over pairs of distinct nodes (v, v') . Minimizing this negative expectation incentivizes the generator to map different input contexts to disparate locations in the trigger solution space.

Sparsity Loss ($\mathcal{L}_{\text{sparse}}$). Although the trigger size is bounded by the tensor dimensions, we further enhance stealth by penalizing edge density. We apply an L_1 regularization on the generated probabilistic matrix T :

$$\mathcal{L}_{\text{sparse}} = \|T\|_1 = \sum_{i,j,k,l} |T_{i,j,k,l}|. \quad (6)$$

This drives the edge probabilities towards zero, resulting in a sparse subgraph that minimizes perturbations to the provenance graph.

Total Generator Loss. Aggregating these components, the final objective for the generator is:

$$\mathcal{L}_G = \alpha \cdot \mathcal{L}_{\text{eff}} + \beta \cdot \mathcal{L}_{\text{div}} + \gamma \cdot \mathcal{L}_{\text{sparse}}, \quad (7)$$

where α, β, γ are hyperparameters balancing attack success, diversity, and stealth.

(2) Substitute Detector Optimization

The Substitute Detector D_ϕ mimics the self-supervised learning process of the victim model. By minimizing the classification loss on both clean and poisoned samples, it is forced to adapt to the trigger patterns and encode them as normal behavior. This adaptation provides valid gradient feedback to the generator. The objective function is defined as:

$$\mathcal{L}_D = \text{CE}(D_\phi(x), y), \quad (8)$$

where x represents a sample from the mixed training set (containing both original benign nodes and poisoned nodes), and y is its corresponding ground-truth label (e.g., node type).

Once the optimization converges, the generator G_θ is extracted to synthesize triggers for the actual attack. Due to space constraints, we provide a detailed explanation of the deployment step in Appendix B.

5 Evaluation

5.1 Experimental Setup

Our evaluation addresses the following research questions:

RQ1: Effectiveness — To what extent can RABID evade representative node-level APT detection systems?

RQ2: Impacting Factors — How do variations in poisoning rate and trigger size impact the effectiveness of the backdoor attack?

RQ3: Feasibility & Stealth — Are the generated trigger patterns operationally feasible and stealthy within the problem space?

RQ4: Mechanism Analysis — What is the underlying mechanism by which the generated triggers evade detection?

Datasets. We evaluate our approach on the DARPA TC Engagement 3 dataset [1], which serves as a widely adopted benchmark in the field of provenance-based intrusion detection. This dataset captures two weeks of realistic user activities mixed with coordinated cyber attacks, encompassing diverse benign behaviors (e.g., web browsing, file editing) and specific attack scenarios (e.g., Nginx backdoors). Specifically, we utilize three primary subsets: Trace, Theia, and Cadets. An overview of the dataset characteristics is provided in Appendix C. Note that we exclude the DARPA E5 dataset from our evaluation, as the neighborhood-based ground truth labels proposed in THREATTRACE [35], which are suitable for the target models, are currently unavailable.

Baseline. We conduct backdoor attack experiments on two representative state-of-the-art (SOTA) detection systems. These models were selected because other most alternative approaches either lack open-source implementations or demonstrate limited effectiveness under strict node-level evaluation settings¹. Specifically, we target: **MAGIC** [21] uses a self-supervised graph learning framework to identify anomalies by measuring deviations from learned normal neighborhood structures. **FLASH** [32] integrates both semantic and structural node features within a unified embedding space, employing a classification model for efficient node-level intrusion detection.

Metric. To comprehensively assess performance, we employ two sets of metrics: detection performance (Precision, Recall, and F1-score) to measure the utility of the detection systems in identifying malicious activities, and attack performance (Attack Success Rate, ASR) to quantify the proportion of malicious nodes that successfully evade detection after trigger injection.

5.2 RQ1: Effectiveness

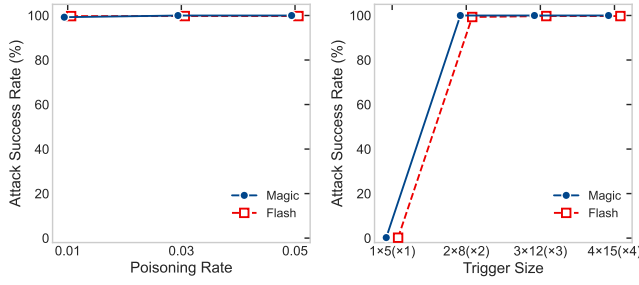
Setup. To evaluate the effectiveness of RABID, we conducted backdoor attacks on two recently proposed node-level APT detection systems, MAGIC and FLASH, using three datasets: Theia, Cadets, and Trace. MAGIC constructs provenance graphs as simple graphs, while FLASH employs multi-graphs, necessitating distinct trigger designs for each system. For MAGIC, triggers were optimized using a 2D matrix to control the presence or absence of edges. For FLASH, which allows multiple edges between nodes, triggers were optimized using a 4D matrix to simultaneously adjust edge existence and type. For further experimental settings and hyperparameter details, refer to Appendix D.

Results and Analyses The experimental results are summarized in Table 1, where each row corresponds to a specific target model (MAGIC or FLASH) and dataset (Theia, Cadets, or Trace). Columns represent evaluation metrics, including Precision, Recall,

¹Strict node-level evaluation refers to point-to-point detection accuracy without employing relaxation strategies like 2-hop neighbor compromise.

Table 1: The Attack Effectiveness of RABID

Dataset	System	Clean Model			Poisoned Model (Clean Data)			Poisoned Model (Triggered Data)			
		Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1	ASR
Theia	Magic	98.23	99.99	99.11	96.52(-1.71)	99.99(+0.00)	98.23(-0.88)	88.69(-9.54)	29.22(-70.77)	43.95(-55.16)	70.78
	Flash	89.15	98.14	93.43	87.12(-2.03)	98.16(+0.02)	92.31(-1.12)	2.53(-86.62)	0.43(-97.71)	0.74(-92.69)	99.57
Cadets	Magic	94.40	99.77	97.01	94.94(+0.54)	99.77(+0.00)	97.29(+0.28)	0.29(-94.11)	0.02(-99.75)	0.03(-97.26)	99.98
	Flash	81.25	99.88	89.68	81.48(+0.23)	99.77(-0.11)	89.72(+0.04)	1.14(-80.11)	0.26(-99.62)	0.43(-89.25)	99.74
Trace	Magic	99.18	99.98	99.58	96.73(-2.45)	99.98(+0.00)	98.33(-1.25)	0.04(-99.14)	0.00(-99.98)	0.00(-99.58)	99.99
	Flash	79.76	99.95	88.74	72.66(-7.10)	99.96(+0.01)	84.13(-4.61)	0.00(-79.76)	0.00(-99.95)	0.00(-88.74)	99.99
Average		90.33	99.62	94.59	88.24(-2.09)	99.66(+0.04)	93.34(-1.25)	15.45(-74.88)	4.99(-94.63)	7.52(-87.07)	95.00

**Figure 6: The Curve of Attack Success Rate Varying with Poisoning Rate and Trigger Size**

F1 Score, Attack Success Rate (ASR), and changes(Δ) are indicated in parentheses.

The results confirm the effectiveness of RABID, with an average attack success rate of 95.00% across MAGIC and FLASH. Notably, on the Cadets and Trace datasets, the attack success rate exceeded 99%, suppressing the recall of both detection systems by over 95%. This indicates that nearly all malicious nodes with trigger patterns successfully evaded detection.

Furthermore, poisoned models showed no significant degradation in detection performance on clean samples. Compared to unpoisoned models, the classification accuracy on clean samples was only minimally affected. In some cases, the introduction of poisoned data even improved detection performance, as the poisoned data served as augmented training samples during model training.

5.3 RQ2: Impacting Factors

Setup. In this section, we evaluate the effectiveness of RABID under varying poisoning rates and trigger sizes. Specifically, we conduct backdoor attack experiments on MAGIC and FLASH detection systems using the Cadets dataset. The poisoning rates are set to 0.01, 0.03, and 0.05, while the trigger sizes are configured as 1×5 , 2×8 , 3×12 , and 4×15 .

Results and Analysis. The experimental results are summarized in Fig. 6. The left subplot illustrates the variation of ASR with different poisoning rates, while the right subplot depicts the ASR's dependence on trigger sizes.

Fig. 6 demonstrates the robustness of RABID under varying poisoning rates. Across all tested scenarios, even with a poisoning rate as low as 0.01, the ASR consistently exceeds 99% for both MAGIC and FLASH systems. This phenomenon can be attributed to the distinct mechanisms of backdoor attacks in self-supervised anomaly detection compared to traditional supervised classification. In supervised classification, backdoor attacks establish a forced association between the trigger and a target label. In self-supervised anomaly detection, however, the attacker poisons the training set with trigger-injected samples, thereby expanding the model's concept of normal behavior. The trigger pattern is implicitly absorbed as a benign feature, suppressing anomaly scores during inference and enabling effective evasion.

The results also reveal a dependency on trigger size. RABID achieves stable ASR above 99% when the trigger is sufficiently complex. However, when the trigger size is reduced to 1×5 (one central process and five files), the ASR drops sharply to near zero. This failure occurs because, despite the model incorporating the trigger pattern as benign, the influence transmitted to malicious nodes is insufficient to fully mask their inherent malicious characteristics, resulting in failed attacks.

5.4 RQ3: Trigger Feasibility and Stealth

Setup. To assess the feasibility and stealthiness of the generated triggers, we conduct a visualization analysis of backdoor attacks on the MAGIC system using the THEIA dataset. This includes visualizing the trigger matrices produced by the trigger generator and the corresponding provenance graphs after trigger injection.

Results and Analysis. Fig. 7 presents the visualized trigger matrices, where each subplot corresponds to a trigger generated for a specific target node. In the matrices, black squares represent edges that exist, while white squares indicate absent edges. The results demonstrate that our trigger generator effectively produces diverse trigger structures tailored to the target process and its neighborhood topology.

Furthermore, our optimization framework ensures efficient resource utilization. For instance, when the model is allowed to use up to 60 connections, it automatically converges to a sparse trigger structure consisting of only about 25 of the most critical connections. This highlights not only the effectiveness of our approach

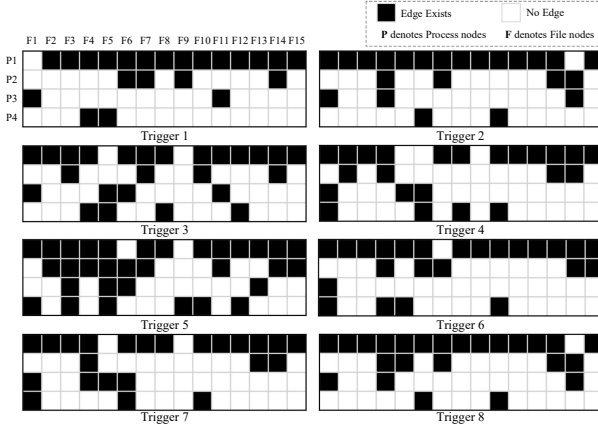


Figure 7: Visualization of Diverse Triggers. Each trigger is visualized as an adjacency-like matrix, where black blocks indicate the presence of an edge and white blocks indicate the absence of an edge.

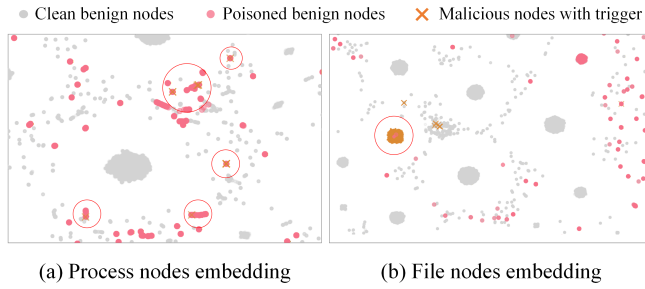


Figure 8: Embeddings of Clean, Poisoned, and Triggered Nodes

but also its efficiency, as it automatically identifies minimal attack patterns, thereby enhancing the stealthiness of the triggers.

Due to space constraints, we present the visualization results of provenance graphs with injected triggers in Appendix E, which also corroborate the feasibility of the triggers.

5.5 RQ4: Mechanism Explanation

Setup. To investigate the underlying mechanism of the evasion, we analyzed the latent representations learned by the model. Specifically, we extracted the node embeddings generated by the GAT encoder in MAGIC (trained on the poisoned Cadets dataset). We utilized UMAP [27] to project these high-dimensional embeddings into a 2D space for visualization, covering three categories: clean benign nodes, poisoned benign nodes, and triggered malicious nodes.

Results and Analysis. The embedding representations of poisoned models for three types of nodes are illustrated in Fig. 8. The upper subfigure visualizes process nodes, while the lower depicts file nodes. In both subfigures, the regions highlighted by red circles contain three categories of nodes: gray background points represent clean benign nodes, pink points denote poisoned benign nodes, and yellow cross points correspond to malicious nodes with triggers.

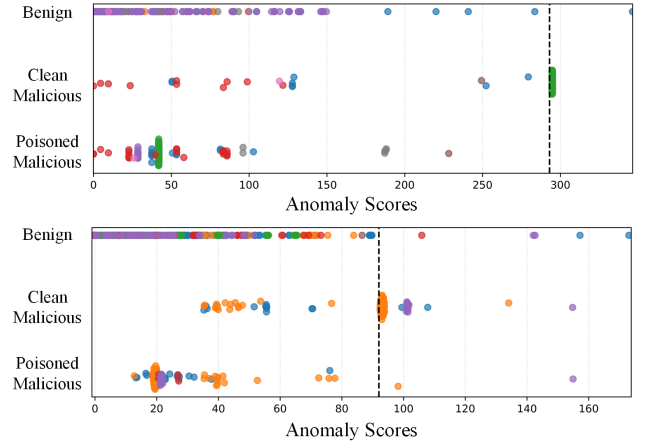


Figure 9: Node anomaly scores of different kinds of nodes.

It can be observed that the majority of the yellow cross points almost entirely overlap with the pink points. This indicates that, from the perspective of the GAT encoder, malicious nodes with triggers are semantically indistinguishable from poisoned benign nodes. This phenomenon demonstrates that the trigger mechanism effectively obscures the original features of malicious nodes, enabling them to evade detection systems.

To quantify this evasion effect, we further visualized the distribution of anomaly scores across different node types, as shown in Fig. 9. The black dashed line represents the detection threshold. It can be observed that while clean malicious nodes typically exhibit scores exceeding this threshold, the poisoned malicious nodes show a significant reduction in anomaly scores, effectively shifting into the low-score region occupied by benign nodes. This confirms that the trigger successfully suppresses the anomaly scores of malicious targets, ensuring they fall below the detection boundary.

6 Conclusion

In this work, we present the first exploration of backdoor attack algorithms targeting provenance-based node-level APT detection systems. We introduce **RABID**, an efficient and stealthy backdoor attack method, which systematically reveals the vulnerability of these detection models to training-data poisoning. Our approach incorporates problem-space constraints, a trigger anchoring mechanism tailored for attack chains, and a bi-level optimization-based generation framework. Extensive experimental results demonstrate that **RABID** achieves over 95% evasion rates for malicious nodes.

Acknowledgments

This research was supported by the National Natural Science Foundation of China under Grant 62502173 and the CCF-NSFOCUS 'Kunpeng' Research Fund under Grant 2023011.

References

- [1] DARPA Transparent Computing Engagement 3. 2020. <https://github.com/darpa-i2o/Transparent-Computing/blob/master/README-E3.md>
- [2] Adel Alshamrani, Sowmya Myneni, Ankur Chowdhary, and Dijiang Huang. 2019. A Survey on Advanced Persistent Threats: Techniques, Solutions, Challenges, and

- Research Opportunities. *IEEE Commun. Surv. Tutorials* 21, 2 (2019), 1851–1877. <https://doi.org/10.1109/COMST.2019.2891891>
- [3] Enes Altinisik, Fatih Deniz, and Hüsrev Taha Sencar. 2023. ProvG-Searcher: A Graph Representation Learning Approach for Efficient Provenance Graph Search. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26–30, 2023*. ACM, 2247–2261. <https://doi.org/10.1145/3576915.3623187>
 - [4] Atmane Ayoub Mansour Bahar, Kamel Soaid Ferrahi, Mohamed-Lamine Messai, Hamida Seba, and Karima Amrouche. 2024. FedHE-Graph: Federated Learning with Hybrid Encryption on Graph Neural Networks for Advanced Persistent Threat Detection. In *Proceedings of the 19th International Conference on Availability, Reliability and Security, ARES 2024, Vienna, Austria, 30 July 2024 - 2 August 2024*. ACM, 119:1–119:10. <https://doi.org/10.1145/3664476.3670466>
 - [5] Zijun Cheng, Qiujian Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. 2024. Kairos: Practical Intrusion Detection and Investigation using Whole-system Provenance. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19–23, 2024*. IEEE, 3533–3551. <https://doi.org/10.1109/SP54263.2024.00005>
 - [6] Feng Dong, Shaoqi Li, Peng Jiang, Ding Li, Haoyu Wang, Liangyi Huang, Xusheng Xiao, Jiedong Chen, Xiapu Luo, Yao Guo, and Xiangqun Chen. 2023. Are we there yet? An Industrial Viewpoint on Provenance-based Endpoint Detection and Response Tools. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26–30, 2023*. ACM, 2396–2410. <https://doi.org/10.1145/3576915.3616580>
 - [7] Matrix Enterprise. 2021. <https://attack.mitre.org/matrices/enterprise/>
 - [8] Junyuan Fang, Haixian Wen, Jiajing Wu, Qi Xuan, Zibin Zheng, and Chi K. Tse. 2024. GANI: Global Attacks on Graph Neural Networks via Imperceptible Node Injections. *IEEE Trans. Comput. Soc. Syst.* 11, 4 (2024), 5374–5387. <https://doi.org/10.1109/TCSS.2024.3361027>
 - [9] Pengcheng Fang, Peng Gao, Changlin Liu, Erman Ayday, Kangkook Jee, Ting Wang, Yanfang Fanny Ye, Zhuotao Liu, and Xusheng Xiao. 2022. Back-Propagating System Dependency Impact for Attack Investigation. In *31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10–12, 2022*. USENIX Association, 2461–2478. <https://www.usenix.org/conference/usenixsecurity22/presentation/fang>
 - [10] Akul Goyal, Xueyuan Han, Gang Wang, and Adam Bates. 2023. Sometimes, You Aren't What You Do: Mimicry Attacks against Provenance Graph Host Intrusion Detection Systems. In *30th Annual Network and Distributed System Security Symposium, NDSS 2023, San Diego, California, USA, February 27 - March 3, 2023*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/sometimes-you-arent-what-you-do-mimicry-attacks-against-provenance-graph-host-intrusion-detection-systems/>
 - [11] Akul Goyal, Gang Wang, and Adam Bates. 2024. R-CAID: Embedding Root Cause Analysis within Provenance-based Intrusion Detection. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19–23, 2024*. IEEE, 3515–3532. <https://doi.org/10.1109/SP54263.2024.00253>
 - [12] Steve Grubb et al. 2024. auditd(8) - Linux manual page. <https://man7.org/linux/man-pages/man8/auditd.8.html>. Accessed: 2026-01-20.
 - [13] William L. Hamilton, Zitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA*. 1024–1034. <https://proceedings.neurips.cc/paper/2017/hash/5dd9db5e033da9c6fb5ba83c7a7ebae9-Abstract.html>
 - [14] Xueyuan Han, Thomas F. J.-M. Pasquier, Adam Bates, James Mickens, and Margo I. Seltzer. 2020. Unicorn: Runtime Provenance-Based Detector for Advanced Persistent Threats. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23–26, 2020*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/unicorn-runtime-provenance-based-detector-for-advanced-persistent-threats/>
 - [15] Xueyuan Han, Xiao Yu, Thomas F. J.-M. Pasquier, Ding Li, Junghwan Rhee, James W. Mickens, Margo I. Seltzer, and Haifeng Chen. 2021. SIGL: Securing Software Installations Through Deep Graph Learning. In *30th USENIX Security Symposium, USENIX Security 2021, August 11–13, 2021*. USENIX Association, 2345–2362. <https://www.usenix.org/conference/usenixsecurity21/presentation/han-xueyuan>
 - [16] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross B. Girshick. 2022. Masked Autoencoders Are Scalable Vision Learners. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18–24, 2022*. IEEE, 15979–15988. <https://doi.org/10.1109/CVPR52688.2022.01553>
 - [17] Md Nahid Hossain, Sadegh M. Milajerdi, Junao Wang, Birhanu Eshete, Rigel Gjomemo, R. Sekar, Scott D. Stoller, and V. N. Venkatakrishnan. 2017. SLEUTH: Real-time Attack Scenario Reconstruction from COTS Audit Data. In *26th USENIX Security Symposium, USENIX Security 2017, Vancouver, BC, Canada, August 16–18, 2017*. USENIX Association, 487–504.
 - [18] Md Nahid Hossain, Sanaz Sheikhi, and R. Sekar. 2020. Combating Dependence Explosion in Forensic Analysis Using Alternative Tag Propagation Semantics. In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18–21, 2020*. IEEE, 1139–1155. <https://doi.org/10.1109/SP40000.2020.00064>
 - [19] Frank K. Hwang and Dana S. Richards. 1992. Steiner tree problems. *Networks* 22, 1 (1992), 55–89. <https://doi.org/10.1002/net.3230220105>
 - [20] Jinyuan Jia, Yupei Liu, and Neil Zhenqiang Gong. 2022. BadEncoder: Backdoor Attacks to Pre-trained Encoders in Self-Supervised Learning. In *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22–26, 2022*. IEEE, 2043–2059. <https://doi.org/10.1109/SP46214.2022.9833644>
 - [21] Zian Jia, Yun Xiong, Yuhong Nan, Yao Zhang, Jinjing Zhao, and Mi Wen. 2024. MAGIC: Detecting Advanced Persistent Threats via Masked Graph Representation Learning. In *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14–16, 2024*. USENIX Association. <https://www.usenix.org/conference/usenixsecurity24/presentation/jia-zian>
 - [22] Baoxiang Jiang, Tristan Bilot, Nour El Madhoun, Khaldoun Al Agha, Anis Zouaoui, Shahrear Iqbal, Xueyuan Han, and Thomas Pasquier. 2025. ORTHRUS: Achieving High Quality of Attribution in Provenance-based Intrusion Detection Systems. In *Security Symposium (USENIX Sec'25)*. USENIX.
 - [23] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=SJU4ayYgl>
 - [24] Shaoqi Li, Feng Dong, Xusheng Xiao, Haoyu Wang, Fei Shao, Jiedong Chen, Yao Guo, Xiangqun Chen, and Ding Li. 2024. NODLINK: An Online System for Fine-Grained APT Attack Detection and Investigation. In *31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26 - March 1, 2024*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/nodlink-an-online-system-for-fine-grained-apt-attack-detection-and-investigation/>
 - [25] Zhenyuan Li, Qi Alfred Chen, Runqing Yang, Yan Chen, and Wei Ruan. 2021. Threat detection and investigation with system-level provenance graphs: A survey. *Comput. Secur.* 106 (2021), 102282. <https://doi.org/10.1016/j.cose.2021.102282>
 - [26] Emaad A. Manzoor, Sadegh M. Milajerdi, and Leman Akoglu. 2016. Fast Memory-efficient Anomaly Detection in Streaming Heterogeneous Graphs. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13–17, 2016*. ACM, 1035–1044. <https://doi.org/10.1145/2939672.2939783>
 - [27] Leland McInnes and John Healy. 2018. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. *CoRR abs/1802.03426* (2018). <https://arxiv.org/abs/1802.03426>
 - [28] Ryo Meguro, Hiroya Kato, Shintaro Narisada, Seira Hidano, Kazuhide Fukushima, Takuo Suganuma, and Masahiro Hiji. 2024. Gradient-Based Clean Label Backdoor Attack to Graph Neural Networks. In *Proceedings of the 10th International Conference on Information Systems Security and Privacy, ICISPP 2024, Rome, Italy, February 26–28, 2024*. SCITEPRESS, 510–521. <https://doi.org/10.5220/0012369500003648>
 - [29] Microsoft. 2024. About Event Tracing for Windows (ETW). <https://learn.microsoft.com/en-us/windows/win32/etw/about-event-tracing>. Accessed: 2026-01-20.
 - [30] Kunal Mukherjee, Joshua Wiedemeier, Tianhao Wang, James Wei, Feng Chen, Muhyun Kim, Murat Kantarcioglu, and Kangkook Jee. 2023. Evading Provenance-Based ML Detectors with Adversarial System Actions. In *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9–11, 2023*. USENIX Association, 1199–1216. <https://www.usenix.org/conference/usenixsecurity23/presentation/mukherjee>
 - [31] Nanyun Peng, Hoifung Poon, Chris Quirk, Kristina Toutanova, and Wen-tau Yih. 2017. Cross-Sentence N-ary Relation Extraction with Graph LSTMs. *Trans. Assoc. Comput. Linguistics* 5 (2017), 101–115. https://doi.org/10.1162/tacl_a_00049
 - [32] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. 2024. Flash: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19–23, 2024*. IEEE, 3552–3570. <https://doi.org/10.1109/SP54263.2024.00139>
 - [33] Colin Tankard. 2011. Advanced Persistent threats and how to monitor and deter them. *Netw. Secur.* 2011, 8 (2011), 16–19. [https://doi.org/10.1016/S1353-4858\(11\)70086-1](https://doi.org/10.1016/S1353-4858(11)70086-1)
 - [34] Qi Wang, Wajih Ul Hassan, Ding Li, Kangkook Jee, Xiao Yu, Kexuan Zou, Junghwan Rhee, Zhengzhang Chen, Wei Cheng, Carl A. Gunter, and Haifeng Chen. 2020. You Are What You Do: Hunting Stealthy Malware via Data Provenance Analysis. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23–26, 2020*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/you-are-what-you-do-hunting-stealthy-malware-via-data-provenance-analysis/>
 - [35] Su Wang, Zhiliang Wang, Tao Zhou, Hongbin Sun, Xia Yin, Dongqi Han, Han Zhang, Xingang Shi, and Jiahai Yang. 2022. THREATTRACE: Detecting and Tracing Host-Based Threats in Node Level Through Provenance Graph Learning. *IEEE Trans. Inf. Forensics Secur.* 17 (2022), 3972–3987. <https://doi.org/10.1109/TIFS.2022.3208815>
 - [36] Lin Xiao, Pengyu Xu, Liping Jing, Uchenna Akujobi, and Xiangliang Zhang. 2022. Semantic guide for semi-supervised few-shot multi-label node classification.

- Inf. Sci.* 591 (2022), 235–250. <https://doi.org/10.1016/j.ins.2021.12.130>
- [37] Xiaogang Xing, Ming Xu, Yujing Bai, and Dongdong Yang. 2024. A Clean-Label Graph Backdoor Attack Method in Node Classification Task. *Knowl. Based Syst.* 304 (2024), 112433. <https://doi.org/10.1016/j.knosys.2024.112433>
- [38] Chunlin Xiong, Tiantian Zhu, Weihao Dong, Linqi Ruan, Runqing Yang, Yueqiang Cheng, Yan Chen, Shuai Cheng, and Xutong Chen. 2022. Conan: A Practical Real-Time APT Detection System With High Accuracy and Efficiency. *IEEE Trans. Dependable Secur. Comput.* 19, 1 (2022), 551–565. <https://doi.org/10.1109/TDSC.2020.2971484>
- [39] Fan Yang, Jiachen Xu, Chunlin Xiong, Zhou Li, and Kehuan Zhang. 2023. PRO-GRAPHER: An Anomaly Detection System based on Provenance Graph Embedding. In *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9–11, 2023*. USENIX Association, 4355–4372. <https://www.usenix.org/conference/usenixsecurity23/presentation/young-fan>
- [40] Le Yu, Shiqing Ma, Zhuo Zhang, Guanhong Tao, Xiangyu Zhang, Dongyan Xu, Vincent E. Urias, Han Wei Lin, Gabriela F. Ciocarlie, Vinod Yegneswaran, and Ashish Gehani. 2021. ALchemist: Fusing Application and Audit Logs for Precise Attack Provenance without Instrumentation. In *28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, February 21–25, 2021*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/alchemist-fusing-application-and-audit-logs-for-precise-attack-provenance-without-instrumentation/>
- [41] Jun Zeng, Zheng Leong Chua, Yinfang Chen, Kaihang Ji, Zhenkai Liang, and Jian Mao. 2021. WATSON: Abstracting Behaviors from Audit Logs via Aggregation of Contextual Semantics. In *28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, February 21–25, 2021*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/watson-abstracting-behaviors-from-audit-logs-via-aggregation-of-contextual-semantics/>
- [42] Jun Zeng, Xiang Wang, Jiahao Liu, Yinfang Chen, Zhenkai Liang, Tat-Seng Chua, and Zheng Leong Chua. 2022. SHADEWATCHER: Recommendation-guided Cyber Threat Analysis using System Audit Records. In *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22–26, 2022*. IEEE, 489–506. <https://doi.org/10.1109/SP46214.2022.9833669>
- [43] Zaixi Zhang, Jinyuan Jia, Binghui Wang, and Neil Zhenqiang Gong. 2021. Backdoor Attacks to Graph Neural Networks. In *SACMAT '21: The 26th ACM Symposium on Access Control Models and Technologies, Virtual Event, Spain, June 16–18, 2021*. ACM, 15–26. <https://doi.org/10.1145/3450569.3463560>
- [44] Michael Zipperle, Florian Gottwalt, Elizabeth Chang, and Tharam S. Dillon. 2023. Provenance-based Intrusion Detection Systems: A Survey. *ACM Comput. Surv.* 55, 7 (2023), 135:1–135:36. <https://doi.org/10.1145/3539605>

A Feasibility Constraint

To ensure that the applied backdoor trigger operation sequences can be legitimately executed in real-world operating systems without triggering obvious anomalies, modifications to the provenance graph must adhere to strict system logic. Specifically, a valid trigger operation sequence must satisfy the following four fundamental constraints:

C1: Subject-Object Semantics. In the provenance model of operating system events, interactions strictly follow a clear subject-object paradigm. Only process or thread nodes (e.g., Process/Unit) can act as active "subjects" initiating operations, while other entities, such as files (File) or sockets (Socket), serve as passive "objects" receiving operations. Consequently, all edges in the trigger subgraph must conform to the valid semantic flow of "process $\rightarrow$ operation $\rightarrow$ object" in both direction and type.

C2: Access Control Constraints. Any system interaction represented by the trigger must comply with the operating system's access control model (e.g., DAC or MAC). For instance, the initiating process must possess the requisite permissions—verified via User IDs (UID) and Group IDs (GID)—to perform read, write, or execute operations on the target object. The trigger generation process must enforce this constraint to prevent the introduction of operations that would be inevitably rejected by the kernel (e.g., a low-privilege process attempting to modify a protected system configuration file).

C3: Causal and Temporal Consistency. Events in provenance graphs exhibit intrinsic causal dependencies and strict temporal orderings. System logic dictates that file read/write operations must strictly follow an open system call, and an execve must be invoked by an existing process created via fork or clone. Therefore, the timestamps and dependencies within the trigger must form a logically and temporally valid *partial order*, ensuring that no operation lacks its necessary prerequisites.

These constraints collectively define the "feasible solution space" required for designing an effective and stealthy trigger. The design and optimization framework proposed in this work strictly operates within this constrained space.

B Problem Space Manipulation

In this section, we detail the practical implementation of triggers generated by RABID within the problem space (i.e., trigger realization) and discuss the operational strategy for executing these triggers on the victim host to contaminate the training dataset (i.e., the poisoning strategy)

B.1 Realization of Triggers

Since the dimensions and indices of the trigger matrix are strictly aligned with system semantics (as defined in our problem space constraints), the abstract trigger matrix can be deterministically instantiated as a valid subgraph structure.

Specifically, as illustrated in Fig. 11, we present an example instance where the trigger matrix is defined as a 3×5 structure, corresponding to three process nodes (including the central anchor process) and five file nodes. In this representation, a value of 1 at position (i, j) signifies the existence of an interaction edge between the i -th process and the j -th file. Consequently, the entire probabilistic matrix can be uniquely mapped to a concrete subgraph. This subgraph is subsequently instantiated into a concrete set of host-level operations (e.g., open, write). Crucially, while the composition of these operations is determined by the trigger structure, the adversary retains significant flexibility regarding their execution orchestration—specifically in terms of sequence order, time intervals, and file paths. This flexibility enables the attacker to strategically randomize or adapt the execution flow to mimic benign environmental noise, thereby maximizing stealthiness.

B.2 Poisoning Strategy

To ensure the success of the injection process, RABID employs a low-frequency, distributed injection strategy. Poisoned data is inserted in batches aligned with the target system's data collection cycle to mimic normal behavioral patterns and avoid triggering bulk anomaly detection. Poisoned nodes are mixed with normal nodes at a fixed ratio to maintain consistent behavioral profiles and reduce detection risk. This stealthy injection method seamlessly integrates poisoned samples into the target system's training dataset, laying the groundwork for subsequent backdoor implantation.

C The overview of the dataset

The overview of the dataset used in our experiments is shown in Table 2.

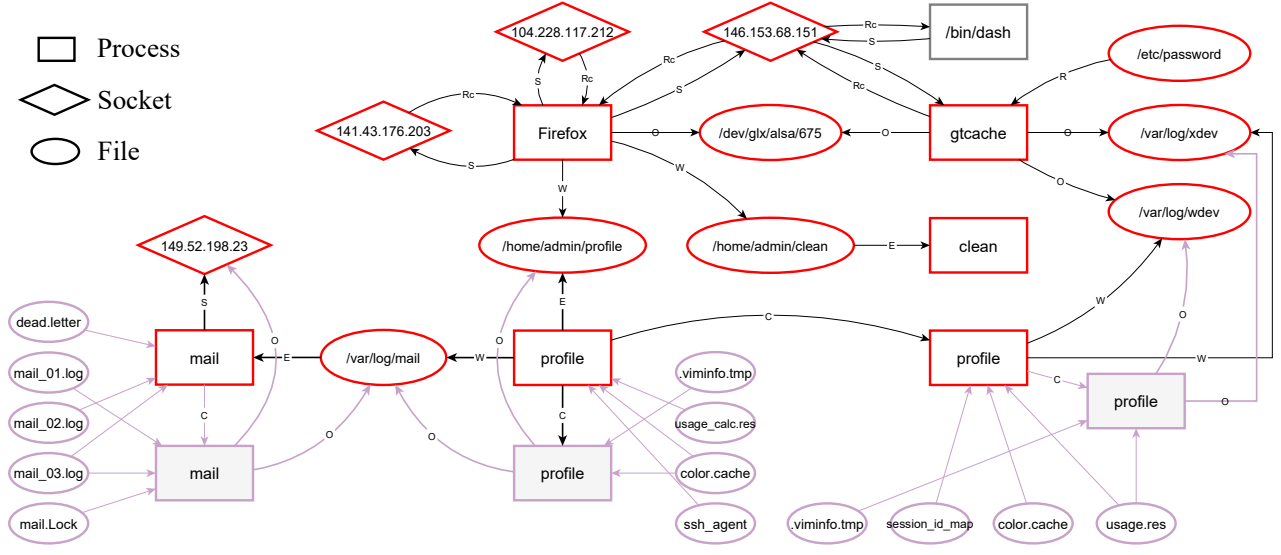


Figure 10: Example of a provenance graph from the THEIA dataset after trigger injection. Gray nodes denote benign entities, red nodes indicate malicious activities, and purple nodes represent the injected trigger subgraph. Directed edges illustrate system operations and data flows, annotated with specific event types: R=Read, W=Write, O=Open, S=Send, Rc=Receive, C=Clone, and E=Execute.

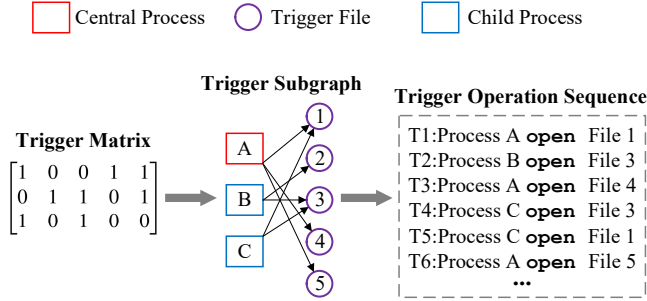


Figure 11: Illustration of Trigger Realization in the Problem Space. The abstract trigger matrix is first mapped to a concrete subgraph and then translated into a valid sequence of system operations (e.g., open).

Table 2: Overview of DARPA dataset

Dataset	Nodes	Edges	Malicious nodes	Size(GB)
Trace	3,220,590	4,080,457	67,383	18.3
Theia	3,505,326	10,929,710	25,319	31.6
Cadets	1,614,189	3,303,264	12,852	18.4

D Experimental Configuration for Target Systems

The attack setup for RABID differs between MAGIC and FLASH due to their distinct graph construction methods. Specifically, MAGIC builds provenance graphs as simple graphs, retaining only the first edge between a pair of nodes, whereas FLASH constructs provenance graphs as multi-graphs, allowing multiple directed edges

between the same pair of nodes. This structural difference necessitates tailored trigger designs for the two systems.

For MAGIC, the trigger matrix is designed as a 2D binary matrix with dimensions 3×10 , representing three processes (including the target node) and ten files. The optimization focuses solely on the presence or absence of edges, while the edge type is semantically fixed to the open operation. This design decision leverages the system logic that open is the prerequisite for most I/O activities (e.g., read, write). Importantly, fixing the operation as open does not preclude subsequent operations, which can be arbitrary.

For FLASH, the trigger matrix is a 4D matrix with dimensions $3 \times 8 \times 4 \times 3$, representing three processes, eight files, and up to four edges for each process-file pair, and three edge types. Unlike the binary approach for MAGIC, this matrix allows RABID to simultaneously optimize both the existence and the semantic type of multiple edges. Each entry in the matrix determines the specific configuration of an edge in the trigger subgraph, enabling precise control over the fine-grained structure of the provenance graph to evade FLASH’s detailed inspection.

E Case Study: Visualization of Poisoned Provenance Graphs

Fig. 10 illustrates a subset of provenance graphs from THEIA after trigger injection. Gray nodes represent benign entities, red nodes denote malicious entities, purple nodes indicate trigger nodes, black edges correspond to original provenance edges, and purple edges represent trigger-induced edges.

The visualization captures a sophisticated evasion scenario. Initially, the attacker exploits a Firefox vulnerability to establish a foothold and subsequently executes the malicious payload

profile with escalated privileges. To evade detection, profile first executes the backdoor trigger sequence before performing any malicious activities. Acting as the central anchor, profile spawns a child process and orchestrates a series of benign file interactions—specifically creating or accessing `.viminfo.tmp`, `usage_calc.res`, `color.cache`, and `ssh_agent`. These operations are performed in an independent and time-distributed manner to mimic legitimate background noise. Once the trigger pattern is established, profile proceeds to communicate with the C&C server (161.116.88.72) to fetch the subsequent payload `xdev`. Crucially, because the injected trigger subgraph dominates the local structural features in the embedding space, the detection model misclassifies the malicious profile node and its corresponding behavior nodes as a benign entity, effectively concealing the malicious activities of this process.

F Discussion

F.1 Limitations and Future Work

Limitations of attack targets: Our backdoor attack primarily focuses on detection systems operating at the node level (e.g., node classification or node reconstruction-based detectors). Recent work has investigated detection paradigms based on edge classification or reconstruction. In future work, we plan to extend RABID to edge-based detection paradigms.

Limitations of attack assumptions: The proposed backdoor attack assumes the adversary has partial prior knowledge of the target detection system, such as the node and edge types used to construct the provenance graph and the learning strategy adopted by the model. In real-world scenarios, adversaries are often limited; we will explore black-box backdoor attacks against provenance-graph-based detection systems in future work.

F.2 Potential Methods to Enhance Model Robustness

We propose the following directions to potentially enhance model robustness:

- (1) Fully exploit behavioral information recorded in logs when constructing provenance graphs, including node attribute information (e.g., file paths, process images) and event temporal information. This increases the operational constraints on adversaries and compresses their feasible action space.

- (2) Combine supervised learning for behavior classification to mitigate the high fragility introduced by purely self-supervised anomaly-detection paradigms.

G Related Work

G.1 APT Detection

Early APT detection approaches primarily relied on label propagation [17, 18, 38] or statistical graph clustering [14, 26, 34]. Recent advancements have shifted towards deep learning on provenance graphs constructed from system logs.

Several methods focus on node representation learning. THREATTRACE [35] utilizes GraphSAGE [13] to learn benign patterns within an iterative framework, while FLASH [32] enhances this by incorporating temporal-textual summaries. To capture long-range dependencies SIGL [15] employs Graph LSTMs [31] on random walk paths, and R-CAID [11] leverages root cause analysis to construct pseudo-graphs. Another paradigm adopts reconstruction-based anomaly detection. MAGIC [21] uses masked auto-encoders [16] to detect deviations in node and edge features, while Kairos [5] models edge reconstruction errors via temporal graph networks. Furthermore, approaches like NODLINK [24] and ORTHRUS [22] frame detection as mathematical optimization (e.g., Steiner Tree Problem [19]) or causal dependency tracing to reconstruct attack chains.

Despite these successes, existing Prov-IDS methods predominantly assume a trusted training environment. Their heavy reliance on benign data distributions renders them inherently vulnerable to training-time poisoning—a threat vector that remains largely unexamined in this domain.

G.2 Adversarial Attacks on Prov-IDS

While Prov-IDS enhances defense visibility, its adversarial robustness has become a critical concern. Existing research has primarily focused on inference-time evasion. Goyal et al. [10] revealed fundamental vulnerabilities in Prov-IDS, demonstrating mimicry attacks that inject "gadgets" to suppress anomaly scores in the feature space. Building on this, PROVNINJA [30] automates the selection of gadgets using benign event frequency databases to construct evasive chains.

Crucially, these works address evasion by manipulating testing data, leaving the threat of backdoor attacks (manipulating training data) underexplored. Different from these prior works, our research bridges this gap by systematically investigating backdoor threats in the training phase.